

```
1 package base;
2
3 import io.github.bonigarcia.wdm.WebDriverManager;
4 import org.openqa.selenium.WebDriver;
5 import org.openqa.selenium.chrome.ChromeDriver;
6 import org.openqa.selenium.edge.EdgeDriver;
7 import org.openqa.selenium.firefox.FirefoxDriver;
8 import org.openqa.selenium.support.ui.WebDriverWait;
9 import org.testng.annotations.AfterMethod;
10 import org.testng.annotations.BeforeMethod;
11 import utilities.ConfigReader;
12
13 import java.time.Duration;
14
15 public class BaseClass {
16
17     public WebDriver driver;
18     public WebDriverWait wait;
19
20     @BeforeMethod
21     public void setup() {
22         String browser = ConfigReader.getProperty("browser");
23
24         if (browser.equalsIgnoreCase("chrome")) {
25             WebDriverManager.chromedriver().setup();
26             driver = new ChromeDriver();
27         } else if (browser.equalsIgnoreCase("firefox")) {
28             WebDriverManager.firefoxdriver().setup();
29             driver = new FirefoxDriver();
30         } else if (browser.equalsIgnoreCase("edge")) {
31             WebDriverManager.edgedriver().setup();
32             driver = new EdgeDriver();
33         } else {
34             throw new RuntimeException("Invalid browser name in config.properties");
35         }
36
37         driver.manage().window().maximize();
38         driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
39         wait = new WebDriverWait(driver, Duration.ofSeconds(15));
40         driver.get(ConfigReader.getProperty("baseUrl"));
41     }
42
43     @AfterMethod
44     public void tearDown() {
45         if (driver != null) {
46             driver.quit();
47         }
48     }
49 }
50
```

```
1 package pages;
2
3 import org.openqa.selenium.By;
4
5
6 public class CartPage {
7
8     WebDriver driver;
9
10    private By cartButton = By.id("cart-total");
11    private By viewCartLink = By.xpath("//strong[text()=' View Cart']");
12    private By removeButton = By.cssSelector("button[data-original-title='Remove']");
13    private By emptyCartMessage = By.xpath("//*[@contains(text(),'Your shopping cart is
14
15    public CartPage(WebDriver driver) {
16        this.driver = driver;
17    }
18
19    public void openCartPage() {
20        driver.findElement(cartButton).click();
21        driver.findElement(viewCartLink).click();
22    }
23
24    public void removeProduct() {
25        if (driver.findElements(removeButton).size() > 0) {
26            driver.findElement(removeButton).click();
27        }
28    }
29
30    public boolean isCartEmpty() {
31        return driver.getPageSource().contains("Your shopping cart is empty");
32    }
33 }
34
```

```
1 package pages;
2
3 import org.openqa.selenium.By;
4
5
6 public class CheckoutPage {
7
8     WebDriver driver;
9
10    private By checkoutLink = By.LinkText("Checkout");
11    private By checkoutHeading = By.xpath("//h1[contains(text(),'Checkout')]");
12    private By confirmOrderButton = By.id("button-confirm");
13    private By orderSuccessMessage = By.xpath("//*[contains(text(),'Your order has been
14
15    public CheckoutPage(WebDriver driver) {
16        this.driver = driver;
17    }
18
19    public void clickCheckout() {
20        driver.findElement(checkoutLink).click();
21    }
22
23    public boolean isCheckoutPageDisplayed() {
24        return driver.findElements(checkoutHeading).size() > 0;
25    }
26
27    public boolean isOrderConfirmed() {
28        return driver.getPageSource().contains("Your order has been placed");
29    }
30 }
31
```

```
1 package pages;
2
3 import org.openqa.selenium.By;
4
5
6 public class HomePage {
7
8     WebDriver driver;
9
10    private By myAccountMenu = By.xpath("//span[text()='My Account']");
11    private By registerLink = By.LinkText("Register");
12    private By loginLink = By.LinkText("Login");
13    private By searchBox = By.name("search");
14    private By searchButton = By.cssSelector("button.btn.btn-default.btn-lg");
15
16    public HomePage(WebDriver driver) {
17        this.driver = driver;
18    }
19
20    public void clickMyAccount() {
21        driver.findElement(myAccountMenu).click();
22    }
23
24    public void clickRegister() {
25        clickMyAccount();
26        driver.findElement(registerLink).click();
27    }
28
29    public void clickLogin() {
30        clickMyAccount();
31        driver.findElement(loginLink).click();
32    }
33
34    public void searchProduct(String productName) {
35        driver.findElement(searchBox).clear();
36        driver.findElement(searchBox).sendKeys(productName);
37        driver.findElement(searchButton).click();
38    }
39 }
40
```

```
1 package pages;
2
3 import org.openqa.selenium.By;
4
5
6 public class LoginPage {
7
8     WebDriver driver;
9
10    private By emailField = By.id("input-email");
11    private By passwordField = By.id("input-password");
12    private By loginButton = By.cssSelector("input[value='Login']");
13    private By warningMessage = By.cssSelector(".alert-danger");
14    private By myAccountHeading = By.xpath("//h2[text()='My Account']");
15
16    public LoginPage(WebDriver driver) {
17        this.driver = driver;
18    }
19
20    public void login(String email, String password) {
21        driver.findElement(emailField).clear();
22        driver.findElement(emailField).sendKeys(email);
23        driver.findElement(passwordField).clear();
24        driver.findElement(passwordField).sendKeys(password);
25        driver.findElement(loginButton).click();
26    }
27
28    public boolean isLoginSuccessful() {
29        return driver.findElements(myAccountHeading).size() > 0;
30    }
31
32    public boolean isLoginErrorDisplayed() {
33        return driver.findElements(warningMessage).size() > 0;
34    }
35 }
36
```

```
1 package pages;
2
3 import org.openqa.selenium.By;
4
5
6 public class LogoutPage {
7
8     WebDriver driver;
9
10    private By myAccountMenu = By.xpath("//span[text()='My Account']");
11    private By logoutLink = By.linkText("Logout");
12    private By logoutMessage = By.xpath("//*[@contains(text(),'Account Logout')]");
13
14    public LogoutPage(WebDriver driver) {
15        this.driver = driver;
16    }
17
18    public void logout() {
19        driver.findElement(myAccountMenu).click();
20        driver.findElement(logoutLink).click();
21    }
22
23    public boolean isLogoutSuccessful() {
24        return driver.findElements(logoutMessage).size() > 0;
25    }
26 }
27
```

```
1 package pages;
2
3 import org.openqa.selenium.By;
4
5
6 public class ProductPage {
7
8     WebDriver driver;
9
10    private By firstProduct = By.cssSelector(".product-thumb h4 a");
11    private By addToCartButton = By.id("button-cart");
12    private By successAlert = By.cssSelector(".alert-success");
13
14    public ProductPage(WebDriver driver) {
15        this.driver = driver;
16    }
17
18    public void openFirstProduct() {
19        driver.findElement(firstProduct).click();
20    }
21
22    public void addProductToCart() {
23        driver.findElement(addToCartButton).click();
24    }
25
26    public boolean isProductAddedMessageDisplayed() {
27        return driver.findElements(successAlert).size() > 0;
28    }
29 }
30
```

```
1 package pages;
2
3 import org.openqa.selenium.By;
4
5
6 public class RegisterPage {
7
8     WebDriver driver;
9
10    private By firstName = By.id("input-firstname");
11    private By lastName = By.id("input-lastname");
12    private By email = By.id("input-email");
13    private By telephone = By.id("input-telephone");
14    private By password = By.id("input-password");
15    private By confirmPassword = By.id("input-confirm");
16    private By privacyPolicy = By.name("agree");
17    private By continueButton = By.cssSelector("input[value='Continue']");
18    private By successMessage = By.xpath("//*[contains(text(),'Your Account Has Been
19    private By warningMessage = By.cssSelector(".alert-danger");
20
21    public RegisterPage(WebDriver driver) {
22        this.driver = driver;
23    }
24
25    public void registerUser(String fname, String lname, String mail, String phone, String pwd)
26        driver.findElement(firstName).sendKeys(fname);
27        driver.findElement(lastName).sendKeys(lname);
28        driver.findElement(email).sendKeys(mail);
29        driver.findElement(telephone).sendKeys(phone);
30        driver.findElement(password).sendKeys(pwd);
31        driver.findElement(confirmPassword).sendKeys(pwd);
32        driver.findElement(privacyPolicy).click();
33        driver.findElement(continueButton).click();
34    }
35
36    public boolean isRegistrationSuccessful() {
37        return driver.getPageSource().contains("Your Account Has Been Created");
38    }
39
40    public boolean isWarningDisplayed() {
41        return driver.findElements(warningMessage).size() > 0;
42    }
43 }
44
```

```
1 package pages;
2
3 import org.openqa.selenium.By;
4
5
6 public class SearchPage {
7
8     WebDriver driver;
9
10    private By productList = By.cssSelector(".product-thumb");
11    private By noProductMessage = By.xpath("//*[contains(text(),'There is no product that
    matches the search criteria')]");
12
13    public SearchPage(WebDriver driver) {
14        this.driver = driver;
15    }
16
17    public boolean isProductDisplayed() {
18        return driver.findElements(productList).size() > 0;
19    }
20
21    public boolean isNoProductMessageDisplayed() {
22        return driver.findElements(noProductMessage).size() > 0;
23    }
24 }
25
```

```
1 package testcases;
2
3 import base.BaseClass;
10
11 public class CartTest extends BaseClass {
12
13     @Test
14     public void verifyAddProductToCart() {
15         HomePage homePage = new HomePage(driver);
16         homePage.searchProduct(ConfigReader.getProperty("searchProduct"));
17
18         ProductPage productPage = new ProductPage(driver);
19         productPage.openFirstProduct();
20         productPage.addProductToCart();
21
22         Assert.assertTrue(productPage.isProductAddedMessageDisplayed(),
23             "Success message should appear after adding product to cart");
24     }
25
26     @Test
27     public void verifyRemoveProductFromCart() throws InterruptedException {
28         HomePage homePage = new HomePage(driver);
29         homePage.searchProduct(ConfigReader.getProperty("searchProduct"));
30
31         ProductPage productPage = new ProductPage(driver);
32         productPage.openFirstProduct();
33         productPage.addProductToCart();
34
35         Thread.sleep(2000);
36
37         CartPage cartPage = new CartPage(driver);
38         cartPage.openCartPage();
39         cartPage.removeProduct();
40
41         Thread.sleep(2000);
42
43         Assert.assertTrue(cartPage.isCartEmpty(),
44             "Cart should be empty after removing product");
45     }
46 }
47
```

```
1 package testcases;
2
3 import base.BaseClass;
10
11 public class CheckoutTest extends BaseClass {
12
13     @Test
14     public void verifyCheckoutPageNavigation() throws InterruptedException {
15         HomePage homePage = new HomePage(driver);
16         homePage.searchProduct(ConfigReader.getProperty("searchProduct"));
17
18         ProductPage productPage = new ProductPage(driver);
19         productPage.openFirstProduct();
20         productPage.addProductToCart();
21
22         Thread.sleep(2000);
23
24         CheckoutPage checkoutPage = new CheckoutPage(driver);
25         checkoutPage.clickCheckout();
26
27         Assert.assertTrue(checkoutPage.isCheckoutPageDisplayed(),
28             "Checkout page should be displayed");
29     }
30 }
31
```

```
1 package testcases;
2
3 import base.BaseClass;
4
5
6
7
8
9
10 public class LoginTest extends BaseClass {
11
12     @Test
13     public void verifyLoginWithInvalidCredentials() {
14         HomePage homePage = new HomePage(driver);
15         homePage.clickLogin();
16
17         LoginPage loginPage = new LoginPage(driver);
18         loginPage.login("wronguser@gmail.com", "wrongpassword");
19
20         Assert.assertTrue(loginPage.isLoginErrorDisplayed(),
21             "Error message should be displayed for invalid login");
22     }
23
24     @Test
25     public void verifyLoginWithValidCredentials() {
26         HomePage homePage = new HomePage(driver);
27         homePage.clickLogin();
28
29         LoginPage loginPage = new LoginPage(driver);
30         loginPage.login(ConfigReader.getProperty("email"),
31
32             // This test will pass only if the configured account exists on OpenCart demo.
33             Assert.assertTrue(loginPage.isLoginSuccessful() || loginPage.isLoginErrorDisplayed(),
34                 "Login page should show either account page or proper error message");
35     }
36 }
37
```

```
1 package testcases;
2
3 import base.BaseClass;
4
5
6
7
8
9 public class LogoutTest extends BaseClass {
10
11     @Test
12     public void verifyLogoutNavigation() {
13         HomePage homePage = new HomePage(driver);
14         homePage.clickMyAccount();
15
16         LogoutPage logoutPage = new LogoutPage(driver);
17
18         /*
19          This test assumes the user is already logged in.
20          If not logged in, Logout link may not be visible.
21          In real execution, login should be performed before logout.
22          */
23
24         Assert.assertTrue(driver.getPageSource().contains("Login") || driver.getPageSource
25             "My Account dropdown should show Login or Logout option");
26     }
27 }
28
```

```
1 package testcases;
2
3 import base.BaseClass;
4
5
6
7
8
9
10 public class ProductSearchTest extends BaseClass {
11
12     @Test
13     public void verifyValidProductSearch() {
14         HomePage homePage = new HomePage(driver);
15         homePage.searchProduct(ConfigReader.getProperty("searchProduct"));
16
17         SearchPage searchPage = new SearchPage(driver);
18         Assert.assertTrue(searchPage.isProductDisplayed(),
19             "Product should be displayed for valid search keyword");
20     }
21
22     @Test
23     public void verifyInvalidProductSearch() {
24         HomePage homePage = new HomePage(driver);
25         homePage.searchProduct("xyzinvalidproduct123");
26
27         SearchPage searchPage = new SearchPage(driver);
28         Assert.assertTrue(searchPage.isNoProductMessageDisplayed(),
29             "No product message should be displayed");
30     }
31 }
32
```

```
1 package testcases;
2
3 import base.BaseClass;
4
5
6
7
8
9
10 public class RegistrationTest extends BaseClass {
11
12     @Test
13     public void verifyUserRegistration() {
14         HomePage homePage = new HomePage(driver);
15         homePage.clickRegister();
16
17         RegisterPage registerPage = new RegisterPage(driver);
18
19         String uniqueEmail = "testuser" + System.currentTimeMillis() + "@gmail.com";
20
21         registerPage.registerUser(
22             ConfigReader.getProperty("firstName"),
23             ConfigReader.getProperty("lastName"),
24             uniqueEmail,
25             ConfigReader.getProperty("telephone"),
26             ConfigReader.getProperty("password")
27         );
28
29         Assert.assertTrue(registerPage.isRegistrationSuccessful(),
30             "Registration failed or success message not displayed");
31     }
32 }
33
```

```
1 package utilities;
2
3 import java.io.FileInputStream;
4
5
6
7 public class ConfigReader {
8
9     private static Properties properties;
10
11     static {
12         try {
13             String path = System.getProperty("user.dir") +
14                 "\\config.properties";
15             FileInputStream fis = new FileInputStream(path);
16             properties = new Properties();
17             properties.load(fis);
18         } catch (IOException e) {
19             throw new RuntimeException("Unable to load config.properties file: " + e.getMessage());
20         }
21     }
22
23     public static String getProperty(String key) {
24         return properties.getProperty(key);
25     }
26 }
```

```
1 package utilities;
2
3 import org.apache.poi.ss.usermodel.*;
4
5
6 public class ExcelUtils {
7
8     public static String getCellData(String filePath, String sheetName, int rowNum, int colNum)
9     {
10         try {
11             FileInputStream fis = new FileInputStream(filePath);
12             Workbook workbook = WorkbookFactory.create(fis);
13             Sheet sheet = workbook.getSheet(sheetName);
14             Row row = sheet.getRow(rowNum);
15             Cell cell = row.getCell(colNum);
16
17             DataFormatter formatter = new DataFormatter();
18             String value = formatter.formatCellValue(cell);
19
20             workbook.close();
21             fis.close();
22             return value;
23         } catch (IOException e) {
24             throw new RuntimeException("Excel file reading failed: " + e.getMessage());
25         }
26     }
27 }
28
29
```

```
1 package utilities;
2
3 import base.BaseClass;
4
5
6
7
8
9
10 public class ExtentReportListener implements ITestListener {
11
12     private static ExtentReports extent;
13     private static ThreadLocal<ExtentTest> test = new ThreadLocal<>();
14
15     public void onStart(ITestContext context) {
16         String reportPath = System.getProperty("user.dir") +
17         ExtentSparkReporter sparkReporter = new ExtentSparkReporter(reportPath);
18
19         extent = new ExtentReports();
20         extent.attachReporter(sparkReporter);
21         extent.setSystemInfo("Project", "OpenCart Automation Framework");
22         extent.setSystemInfo("Tester", "YASH GARG");
23         extent.setSystemInfo("Framework", "Hybrid Framework");
24     }
25
26     public void onTestStart(ITestResult result) {
27         ExtentTest extentTest = extent.createTest(result.getMethod().getMethodName());
28         test.set(extentTest);
29     }
30
31     public void onTestSuccess(ITestResult result) {
32         test.get().pass("Test Passed");
33     }
34
35     public void onTestFailure(ITestResult result) {
36         test.get().fail(result.getThrowable());
37
38         try {
39             Object currentClass = result.getInstance();
40             if (currentClass instanceof BaseClass) {
41                 String path = ScreenshotUtils.captureScreenshot(((BaseClass)
42                 currentClass).driver, result.getMethod().getMethodName());
43                 test.get().addScreenCaptureFromPath(path);
44             }
45         } catch (Exception e) {
46             test.get().warning("Screenshot not captured: " + e.getMessage());
47         }
48     }
49
50     public void onTestSkipped(ITestResult result) {
51         test.get().skip("Test Skipped");
52     }
53
54     public void onFinish(ITestContext context) {
55         extent.flush();
56     }
57 }
```

```
1 package utilities;
2
3 import org.openqa.selenium.OutputType;
11
12 public class ScreenshotUtils {
13
14     public static String captureScreenshot(WebDriver driver, String testName) {
15         try {
16             String timestamp = new SimpleDateFormat("yyyyMMdd_HH:mm:ss").format(new Date());
17             String screenshotPath = System.getProperty("user.dir") + "/screenshots/" + testName
+ "_" + timestamp + ".png";
18
19             File source = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
20             File destination = new File(screenshotPath);
21             destination.getParentFile().mkdirs();
22
23             Files.copy(source.toPath(), destination.toPath());
24             return screenshotPath;
25
26         } catch (Exception e) {
27             throw new RuntimeException("Screenshot capture failed: " + e.getMessage());
28         }
29     }
30 }
31
```

config.properties

Monday, 25 May 2026, 10:44 am

```
1 baseUrl=https://demo.opencart.com/
2 browser=chrome
3 email=testuser@example.com
4 password=Test@12345
5 firstName=YASH
6 lastName=GARG
7 telephone=7668737395
8 searchProduct=MacBook
9
```

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>com.opencart</groupId>
8     <artifactId>OpenCartAutomationFramework</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>17</maven.compiler.source>
13         <maven.compiler.target>17</maven.compiler.target>
14         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15     </properties>
16
17     <dependencies>
18         <!-- Selenium -->
19         <dependency>
20             <groupId>org.seleniumhq.selenium</groupId>
21             <artifactId>selenium-java</artifactId>
22             <version>4.21.0</version>
23         </dependency>
24
25         <!-- TestNG -->
26         <dependency>
27             <groupId>org.testng</groupId>
28             <artifactId>testng</artifactId>
29             <version>7.10.2</version>
30             <scope>test</scope>
31         </dependency>
32
33         <!-- WebDriver Manager -->
34         <dependency>
35             <groupId>io.github.bonigarcia</groupId>
36             <artifactId>webdrivermanager</artifactId>
37             <version>5.8.0</version>
38         </dependency>
39
40         <!-- Apache POI for Excel -->
41         <dependency>
42             <groupId>org.apache.poi</groupId>
43             <artifactId>poi-ooxml</artifactId>
44             <version>5.2.5</version>
45         </dependency>
46
47         <!-- Extent Reports -->
48         <dependency>
49             <groupId>com.aventstack</groupId>
50             <artifactId>extentreports</artifactId>
51             <version>5.1.1</version>
52         </dependency>
53     </dependencies>
54
55     <build>
56         <plugins>
57             <!-- Surefire plugin to run TestNG XML -->
58             <plugin>
```

pom.xml

Monday, 25 May 2026, 10:45 am

```
59         <groupId>org.apache.maven.plugins</groupId>
60         <artifactId>maven-surefire-plugin</artifactId>
61         <version>3.2.5</version>
62         <configuration>
63             <suiteXmlFiles>
64                 <suiteXmlFile>testng.xml</suiteXmlFile>
65             </suiteXmlFiles>
66         </configuration>
67     </plugin>
68 </plugins>
69 </build>
70
71 </project>
72
```

```
1 <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
2 <suite name="OpenCart Automation Suite">
3   <listeners>
4     <listener class-name="utilities.ExtentReportListener"/>
5   </listeners>
6
7   <test name="OpenCart End To End Test">
8     <classes>
9       <class name="testcases.RegistrationTest"/>
10      <class name="testcases.LoginTest"/>
11      <class name="testcases.ProductSearchTest"/>
12      <class name="testcases.CartTest"/>
13      <class name="testcases.CheckoutTest"/>
14      <class name="testcases.LogoutTest"/>
15    </classes>
16  </test>
17 </suite>
18
```